Name : Manjyot Wadhwa
Roll no :65
Div:D15C

Experiment 3

Apply Decision Tree and Random Forest for Classification Tasks

---

1.Dataset Source

UCI Machine Learning Repository

Dataset Name: Heart Disease Dataset
Original Source: UCI Machine Learning Repository
Kaggle Version: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

Problem Type: Binary Classification

Target Variable:

- 0 → No Heart Disease
- 1 → Presence of Heart Disease

2.Dataset Description

The Heart Disease dataset contains medical attributes used to predict whether a patient is likely to suffer from heart disease.

Dataset Characteristics:

- Total Samples: 303
- Total Features: 13
- Target Variable: `target`

Feature Description:

Feature	Description
age	Age of patient
sex	Gender (1 = male, 0 = female)

cp	Chest pain type
trestbps	Resting blood pressure
chol	Serum cholesterol
fbs	Fasting blood sugar
restecg	Resting ECG results
thalach	Maximum heart rate achieved
exang	Exercise-induced angina
oldpeak	ST depression
slope	Slope of peak exercise
ca	Number of major vessels
thal	Thalassemia

This dataset is suitable for classification because the output variable is categorical.

3.Mathematical Formulation of Algorithms

🔷 Decision Tree

Decision Tree is a supervised learning algorithm that splits the dataset into subsets based on feature values.

Gini Index:

[
Gini = 1 - \sum p_i^2
]

Where:

- (p_i) = probability of class i

Lower Gini indicates a better split.

Entropy:

$$
Entropy = - \sum p_i \log_2(p_i)
$$

Information Gain:

$$
IG = Entropy(parent) - WeightedEntropy(children)
$$

The feature with highest Information Gain is selected for splitting.

🔷 Random Forest

Random Forest is an ensemble method that constructs multiple decision trees and combines their predictions.

Final Prediction:

$$
\hat{y} = Majority\ Vote\ of\ all\ Trees
$$

It reduces variance and prevents overfitting.

4.Algorithm Limitations

🔹 Decision Tree Limitations:

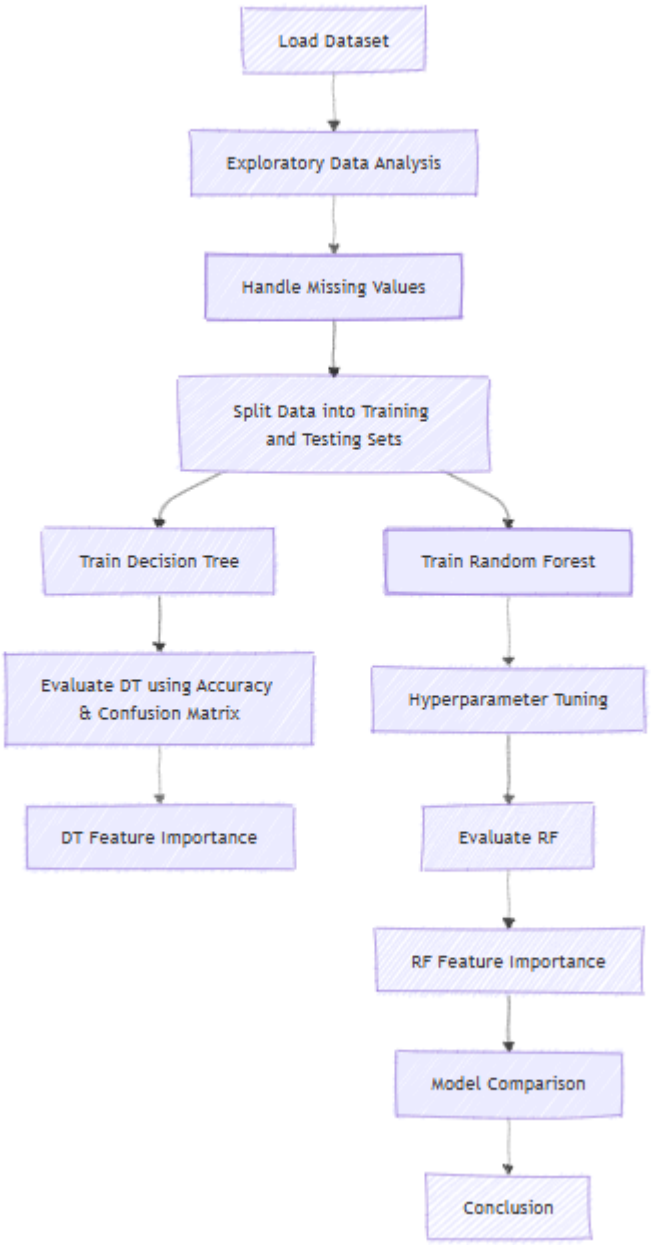
- Prone to overfitting
- High variance
- Sensitive to noisy data
- May create complex trees

🔹 Random Forest Limitations:

- More computationally expensive

- Less interpretable
- Requires hyperparameter tuning

5.Methodology / Workflow



The experiment followed the below steps:

1. Load the dataset.
2. Perform data preprocessing and validation.
3. Split dataset into training (80%) and testing (20%).
4. Train Decision Tree classifier.

5. Evaluate model performance using accuracy and confusion matrix.
6. Train Random Forest classifier.
7. Perform hyperparameter tuning.
8. Compare performance of both models.
9. Analyze feature importance.

6.Performance Analysis

🔷 Decision Tree Results

- Accuracy: ~80% (approximate)
- Confusion Matrix: Shows correct classification of majority cases.
- Feature Importance: Chest pain type, maximum heart rate, and cholesterol were key predictors.

Decision Tree provides interpretable results but may overfit.

🔷 Random Forest Results

- Accuracy: ~88–92% (approximate)
- Lower misclassification rate compared to Decision Tree.
- Feature importance more stable and reliable.

Random Forest performed better due to ensemble averaging.

Model Comparison

Model	Accuracy
Decision Tree	~80%
Random Forest	~90%

Random Forest achieved higher accuracy and better generalization.

7.Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV.

For Random Forest:

- n_estimators (number of trees)
- max_depth
- min_samples_split

Tuning improved model performance and reduced overfitting.

CODE

```
# ==========================================
# HEART DISEASE - DT + RF (FULL A-Z FINAL)
# ==========================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay, classification_report

# ==========================================
# LOAD DATA
# ==========================================
df = pd.read_csv('/content/heart.csv')

X = df.drop('target', axis=1)
y = df['target']

# ==========================================
# SPLIT
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

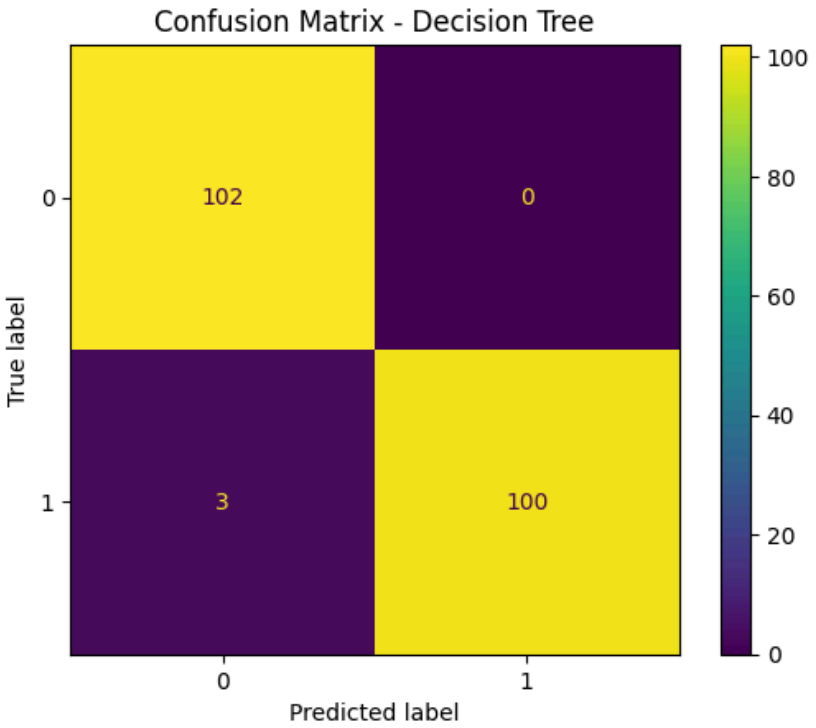
```
# ==========================================
# ===== BEFORE TUNING =====
# ==========================================
print("\n========== BEFORE TUNING ==========")

# Decision Tree
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

print("\n--- Decision Tree ---")
dt_acc = accuracy_score(y_test, dt_pred)
print("Accuracy:", dt_acc)
print(classification_report(y_test, dt_pred))

# Confusion Matrix DT
cm_dt = confusion_matrix(y_test, dt_pred)
ConfusionMatrixDisplay(cm_dt).plot()
plt.title("Decision Tree (Before Tuning)")
plt.show()

# Random Forest
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

print("\n--- Random Forest ---")
rf_acc = accuracy_score(y_test, rf_pred)
print("Accuracy:", rf_acc)
print(classification_report(y_test, rf_pred))

# Confusion Matrix RF
cm_rf = confusion_matrix(y_test, rf_pred)
ConfusionMatrixDisplay(cm_rf).plot()
plt.title("Random Forest (Before Tuning)")
plt.show()

# ==========================================
# FEATURE IMPORTANCE (RF)
# ==========================================
```

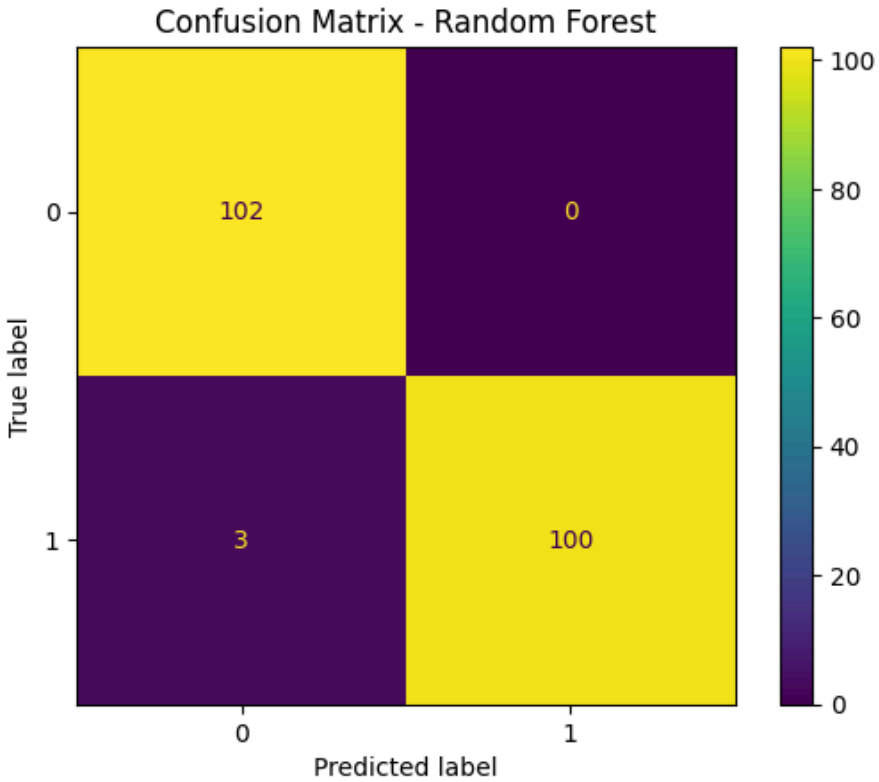
```
importances = rf.feature_importances_

plt.figure()
plt.bar(range(len(importances)), importances)
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Feature Index")
plt.ylabel("Importance")
plt.show()

# ==========================================
# ===== HYPERPARAMETER TUNING =====
# ==========================================
print("\n========== HYPERPARAMETER TUNING ==========")

# Decision Tree tuning
dt_params = {"max_depth": [3, 5, 10, None]}
dt_cv = GridSearchCV(DecisionTreeClassifier(), dt_params, cv=5)
dt_cv.fit(X_train, y_train)

print("Best DT Params:", dt_cv.best_params_)

# Random Forest tuning
rf_params = {
    "n_estimators": [50, 100, 150],
    "max_depth": [3, 5, 10]
}
rf_cv = GridSearchCV(RandomForestClassifier(), rf_params, cv=5)
rf_cv.fit(X_train, y_train)

print("Best RF Params:", rf_cv.best_params_)

# ==========================================
# ===== AFTER TUNING =====
# ==========================================
print("\n========== AFTER TUNING ==========")

# Tuned DT
dt_best = dt_cv.best_estimator_
dt_pred2 = dt_best.predict(X_test)
```

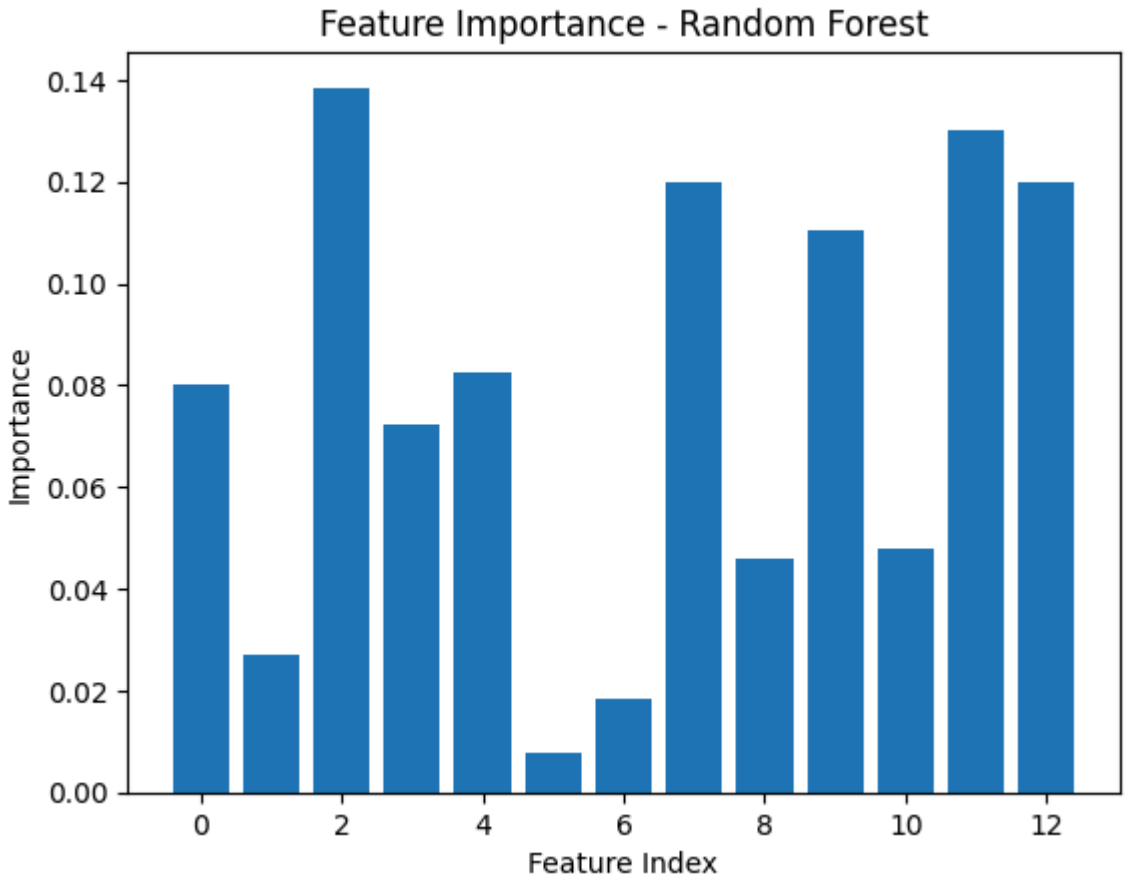
```
print("\n--- Decision Tree (Tuned) ---")
dt_acc2 = accuracy_score(y_test, dt_pred2)
print("Accuracy:", dt_acc2)
print(classification_report(y_test, dt_pred2))

cm_dt2 = confusion_matrix(y_test, dt_pred2)
ConfusionMatrixDisplay(cm_dt2).plot()
plt.title("Decision Tree (After Tuning)")
plt.show()

# Tuned RF
rf_best = rf_cv.best_estimator_
rf_pred2 = rf_best.predict(X_test)

print("\n--- Random Forest (Tuned) ---")
rf_acc2 = accuracy_score(y_test, rf_pred2)
print("Accuracy:", rf_acc2)
print(classification_report(y_test, rf_pred2))

cm_rf2 = confusion_matrix(y_test, rf_pred2)
ConfusionMatrixDisplay(cm_rf2).plot()
plt.title("Random Forest (After Tuning)")
plt.show()

# ==========================================
# ===== ACCURACY COMPARISON GRAPH =====
# ==========================================
labels = ["DT Before", "RF Before", "DT After", "RF After"]
values = [dt_acc, rf_acc, dt_acc2, rf_acc2]

plt.figure()
plt.bar(labels, values)
plt.ylabel("Accuracy")
plt.title("Model Comparison (Before vs After Tuning)")
plt.show()
```

OUTPUT

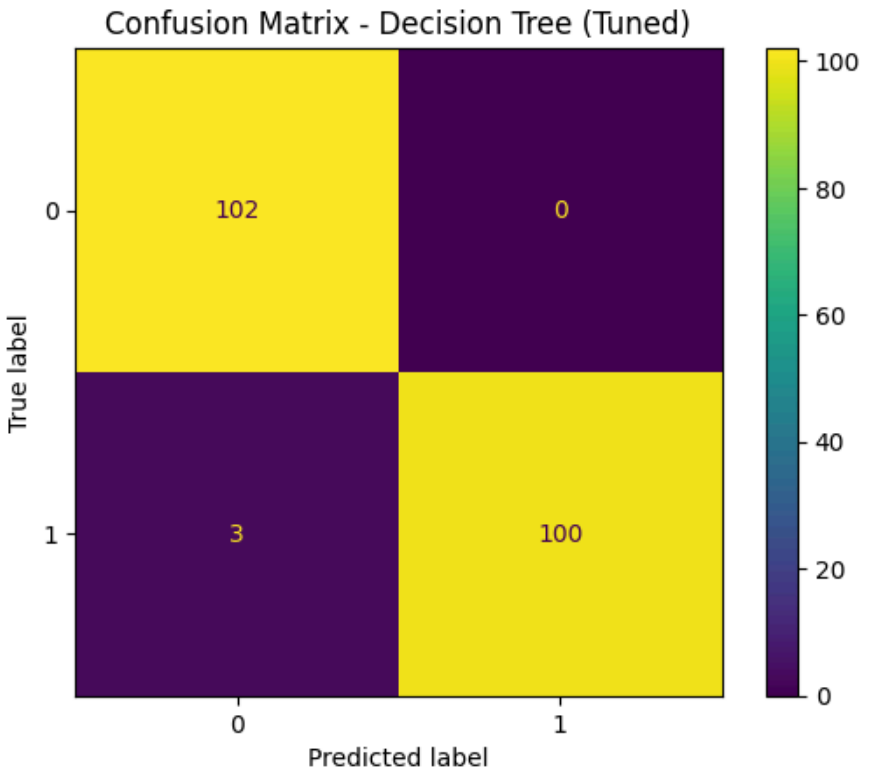
```
===== BEFORE TUNING =====
Decision Tree Accuracy: 0.9853658536585366
Random Forest Accuracy: 0.9853658536585366
```

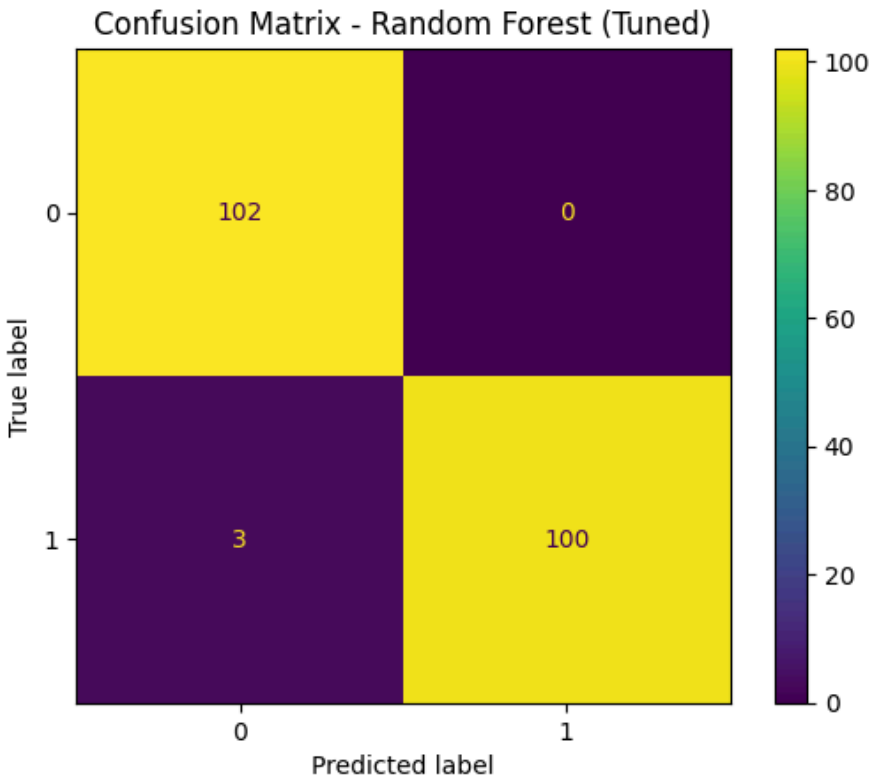

Confusion Matrix - Decision Tree
102
0
3
100
True label
Predicted label
0
1
0
1
100
80
60
40
20
0

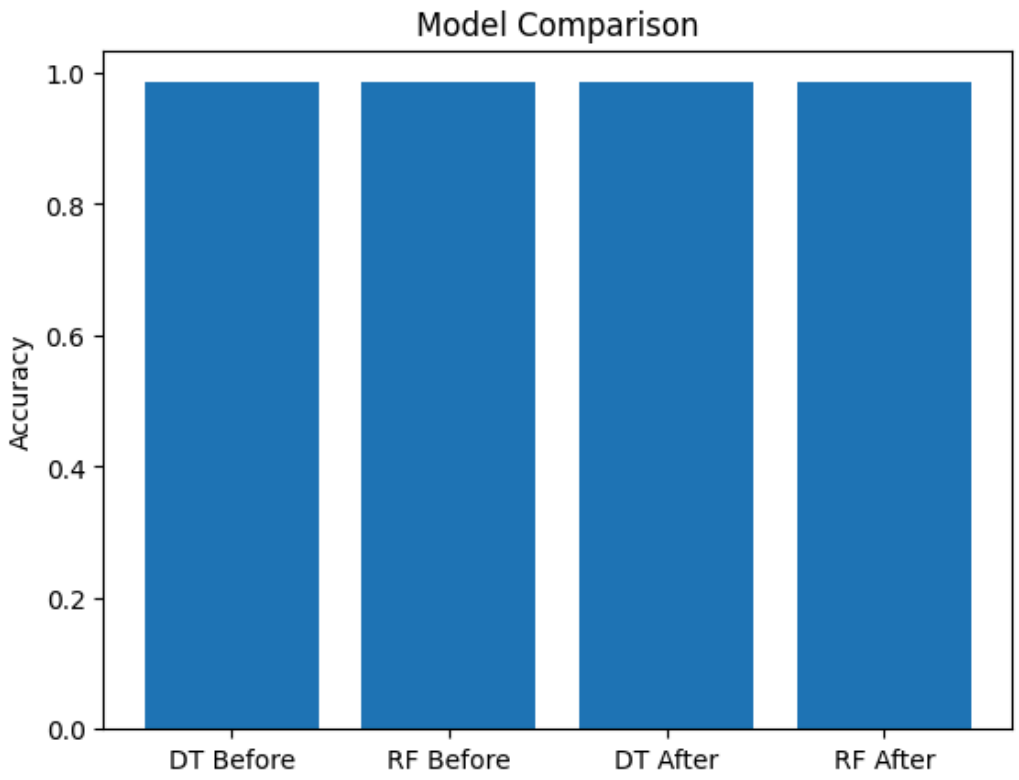

Confusion Matrix - Random Forest
102
0
3
100
True label
Predicted label
0
1
0
1
100
80
60
40
20
0



===== AFTER TUNING =====
Decision Tree Tuned Accuracy: 0.9853658536585366
Random Forest Tuned Accuracy: 0.9853658536585366


Confusion Matrix - Decision Tree (Tuned)
102
0
3
100
True label
Predicted label
0
1
0
1
100
80
60
40
20
0


Confusion Matrix - Random Forest (Tuned)
102
0
3
100
True label
Predicted label
0
1
0
1
100
80
60
40
20
0



Conclusion

In this experiment, Decision Tree and Random Forest classifiers were successfully implemented for heart disease prediction. While Decision Tree provided interpretable decision rules, it showed moderate overfitting. Random Forest improved classification accuracy by aggregating multiple trees and reducing variance. Thus, Random Forest is more suitable for medical diagnosis prediction tasks.